
SOFTWARE ENGINEERING

NOTEBOOK

PRODUCED BY: *vaeVictis*

leo.expats@gmail.com

THIS DOCUMENT IS WRITTEN IN $\text{\LaTeX}$

ENJOY!

Table of Contents

Table of Contents	1
1 Algorithms	4
1.1 Time Complexity	4
1.2 Space Complexity	5
2 Solid Principles	6
2.1 Single-Responsibility	6
2.2 Open-Close	6
2.3 Liskov Substitution	6
2.4 Interface Segregation	7
2.5 Dependency Inversion	7
3 Design Patterns	8
3.1 Creational Patterns	8
3.1.1 Factory Method	8
3.1.2 Abstract Factory	10
3.1.3 Builder	11
3.1.4 Prototype	12
3.1.5 Singleton	14
3.2 Behavioral Patterns	15
3.2.1 Chain of Responsibility	15
3.2.2 Command	16
3.2.3 Iterator	17
3.2.4 Mediator	18
3.2.5 Memento	19
3.2.6 Observer	20
3.2.7 State	21
3.2.8 Strategy	22
3.2.9 Template Method	23
3.2.10 Visitor	23
3.3 Structural Patterns	25
3.3.1 Adapter	25
3.3.2 Bridge	26
3.3.3 Composite	27
3.3.4 Decorator	28
3.3.5 Facade	30
3.3.6 Flyweight	31

3.3.7	Proxy	32
4	Idioms	34
4.1	PImpl Idiom	34
4.2	Fast PImpl Idiom	35
4.3	Handle/Object Idiom	35
4.4	Copy and Swap Idiom	35
4.5	RAII - Resource acquisition is initialization	35
5	Containers	36
5.1	Links to the CPP Reference documentation	36
5.1.1	Sequence Containers	36
5.1.2	Associative Containers	36
5.1.3	Unordered Associative Containers	36
5.1.4	Container Adaptors	36
5.2	Invalidation of Iterators and References	37
6	Memory Management and Smart Pointers	38
6.1	Memory Management	38
6.2	Smart Pointers in C++	39
6.2.1	std::unique_ptr	39
6.2.2	std::shared_ptr	42
6.2.3	std::weak_ptr	43
6.2.4	std::auto_ptr	44
7	C++	45
7.1	C++ Good Practices	45
7.1.1	Rule of three/five/zero	45
7.1.2	Const Correctness	46
7.2	C++11 New Features	48
7.2.1	Value Categories	48
7.2.2	Move Semantic	52
7.2.3	Perfect Forwarding	52
7.2.4	auto	52
7.2.5	decltype	52
7.2.6	const-expr	52
7.3	C++23 New Features	52
7.3.1	Containers	52
8	OOP and UML	54
8.1	Object Oriented Programming	54

8.1.1	Polymorphism	54
8.2	UML	55
8.2.1	Class Diagram	55
9	Concurrency	59
9.1	Threads	59
9.2	Atomic operations	59
9.3	Memory Order	64
10	Dictionary	65
10.1	A	65
10.2	B	66
10.3	C	66
10.4	D	67
10.5	H	68
10.6	I	69
10.7	M	70
10.8	O	71
10.9	P	72
10.10	R	73
10.11	S	74
	References	75
12	References	78

About This File

Algorithms

1.1 Time Complexity

Theta notation

$$f(n) = \Theta(g(n)) \quad \text{if} \quad 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \forall n \geq n_0 \quad (1)$$

For all values $n \geq n_0$, $f(n)$ is equal to $g(n)$ within a constant factor.

$g(n)$ is an asymptotically tight bound for $f(n)$.

Big O notation

$$f(n) = O(g(n)) \quad \text{if} \quad 0 \leq f(n) \leq c g(n) \forall n \geq n_0 \quad (2)$$

For all values $n \geq n_0$, $f(n)$ is on or below $c g(n)$.

A constant multiple of $g(n)$ is an asymptotic upper bound for $f(n)$.

Omega notation

$$f(n) = \Omega(g(n)) \quad \text{if} \quad 0 \leq c g(n) \leq f(n) \forall n \geq n_0 \quad (3)$$

For all values $n \geq n_0$, $f(n)$ is on or above $c g(n)$.

A constant multiple of $g(n)$ is an asymptotic lower bound for $f(n)$.

Meaning of previous notations - Insertion sort case

O-notation describes an upper bound. When we use it to bound the worst-case running time

of an algorithm, we have a bound on the running time of the algorithm on every input. Thus, the $O(n^2)$ bound on worst-case running time of insertion sort also applies to its running time on every input.

The $\Theta(n^2)$ bound on the worst-case running time of insertion sort, however, does not imply a $\Theta(n^2)$ bound on the running time of insertion sort on every input. For example, when the input is already sorted, insertion sort runs in $\Theta(n)$ time. Technically, it is an abuse to say that the running time of insertion sort is $O(n^2)$, since for a given n , the actual running time varies, depending on the particular input of size n . When we say "the running time is $O(n^2)$ ", we mean that there is a function $f(n)$ that is $O(n^2)$ such that for any value of n , no matter what particular input of size n is chosen, the running time on that input is bounded from above by the value $f(n)$. Equivalently, we mean that the worst-case running time is $O(n^2)$.

Omega notation describes a lower bound. When we say that the running time (no modifier) of an algorithm is $\Omega(g(n))$, we mean that no matter what particular input of size n is chosen for each value of n , the running time on that input is at least a constant times $g(n)$, for sufficiently large n . Equivalently, we are giving a lower bound on the best-case running time of an algorithm. For example, the best-case running time of insertion sort is $\Omega(n)$, which implies that the running time of insertion sort is $\Omega(n)$. The running time of insertion sort therefore belongs to both $\Omega(n)$ and $O(n^2)$, since it falls anywhere between a linear function of n and a quadratic function of n . Moreover, these bounds are asymptotically as tight as possible: for instance, the running time of insertion sort is not $\Omega(n^2)$, since there exists an input for which insertion sort runs in $\Theta(n)$ time (e.g., when the input is already sorted). It is not contradictory, however, to say that the worst-case running time of insertion sort is $\Omega(n^2)$, since there exists an input that causes the algorithm to take $\Omega(n^2)$ time.

Methods for solving the recurrence

There are three methods for solving recurrences:

- the substitution method
- the recursion-tree method
- the master theorem

1.2 Space Complexity

Solid Principles

SOLID is a mnemonic acronym for five design principles intended to make object-oriented designs more understandable, flexible, and maintainable.

The principles are a subset of many principles promoted by American software engineer and instructor **Robert C. Martin**, [?]- [?]- [?].

They were first introduced in his 2000 paper Design Principles and Design Patterns, while the acronym was introduced later, around 2004, by **Michael Feathers** [?].

Although the SOLID principles apply to any object-oriented design, they can also form a core philosophy for methodologies such as agile development or adaptive software development.

2.1 Single-Responsibility

"There should never be more than one reason for a class to change." [?]

In other words, every class should have only one responsibility. [?]

2.2 Open-Close

"Software entities ... should be open for extension, but closed for modification." [7]

2.3 Liskov Substitution

"Functions that use pointers or references to base classes must be able to use objects of derived classes without knowing it." [8]

See also: [Design by contract](#).

2.4 Interface Segregation

"Clients should not be forced to depend upon interfaces that they do not use." [9] [?]

2.5 Dependency Inversion

"Depend upon abstractions, [not] concretions." [10] [?]

Design Patterns

Design Patterns describe how to solve recurring design problems to design flexible and reusable object-oriented software, that is, objects that are easier to implement, change, test, and reuse.

3.1 Creational Patterns

3.1.1 Factory Method

This pattern uses **factory methods** to deal with the problem of **creating objects without having to specify the exact class** of the object that will be created.

Objects creation is achieved by calling a factory method — either specified in an interface and implemented by child classes, or implemented in a base class and optionally overridden by derived classes — rather than by calling a constructor.

When to use this pattern

The object's creation may lead to a significant duplication of code, may require information not accessible to the composing object, may not provide a sufficient level of abstraction, or may otherwise not be part of the composing object's concerns.

The factory method design pattern handles these problems by defining a **separate method for creating the objects**, which subclasses can then override to specify the derived type of product that will be created.

Following a list of use cases

- the exact types and dependencies of the objects your code should work with are not known

- there is the need to provide users of a library or framework with a way to extend its internal components
- there is the concern to save system resources by reusing existing objects instead of re-building them each time.

Structure

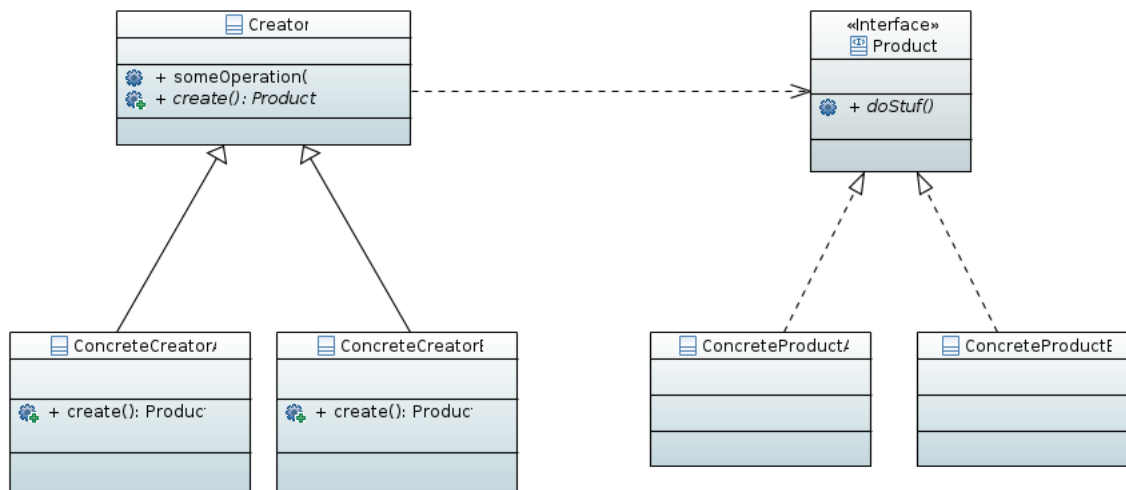


Figure 1: *Factory Method Class Diagram*

The structure of the Factory Method is quite easy

- **Product** defines the interface common to all the objects.
- **Concrete Products** are the different implementations of the previous interface.
- **Creator** is the base class which declares the factory method.
 - **factory method** should either be abstract or return a default product.
 - **returned type** must match the interface
 -
- **Concrete Creators** override the base factory method in order to return different types of **Concrete Products**, which could not be a new instance every time. Factory methods could also return existing objects from a cache, an object in a pool or similar ones.

Factory Method helps decouple the creation of the different concrete objects from the core business logic the Creator might and usually does have.

Pros and Cons

Relation with other Design Patterns

3.1.2 Abstract Factory

This pattern makes it easier to produce families of related items without specifying their concrete classes.

This is achieved declaring **explicit interfaces for each distinct product** of the product family and making **all variants of products need to follow those interfaces**.

An Abstract Factory is finally declared with a **list of creation methods** for all the items that are part of the product family. For each variant of a product family, **a separate factory class** based on the AbstractFactory interface will be needed and **clients access with both factories and products via their abstract interfaces**.

When to use this pattern

-
-
-

Structure

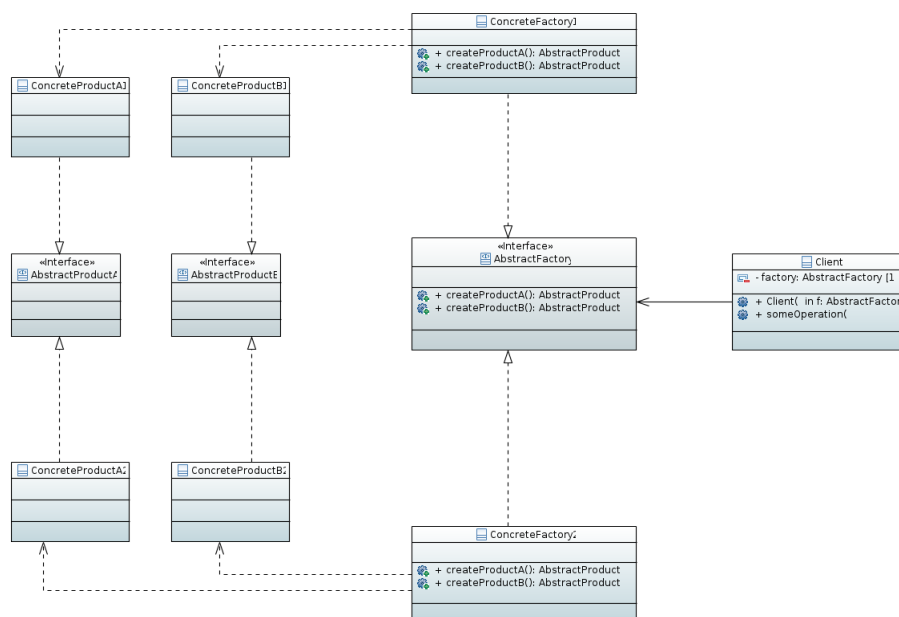


Figure 2: *Abstract Factory Class Diagram*

- **Abstract Products** declare the interfaces for a set or family of related but distinct products.

- **Concrete Products** are the implementations of the concrete variants of abstract products. Each single abstract product must be implemented in all given variants.
- **Abstract Factory** is the interface to collect the declarations of sets of methods to create each of the abstract products.
- **Concrete Factorys** implement the creation methods of the abstract factory, each creating only the product variant it corresponds to.
- **Client** can work with any concrete factory as long as it communicates with the objects through their abstract interfaces. In order to have this behavior, the signatures of the concrete factories' creation methods must match the corresponding abstract interfaces'.

Pros and Cons

Relation with other Design Patterns

3.1.3 Builder

Builder facilitates the **construction of complex objects step by step**, producing different types and representations of an object using the same construyction code. This is achieved extracting the construction code out of its own class and moving it to a separate Builder class. Construction is managed with a set of steps executed in series and only in case they are necessary for producing a specific configuration of the object. A specific Builder class could be implemented to manage the same building steps in a different manner. A Director class, not mandatory, can be implemented to collect the steps in a specific order, while the Builder provides the implementation of what to do.

When to use this pattern

-
-
-

Structure

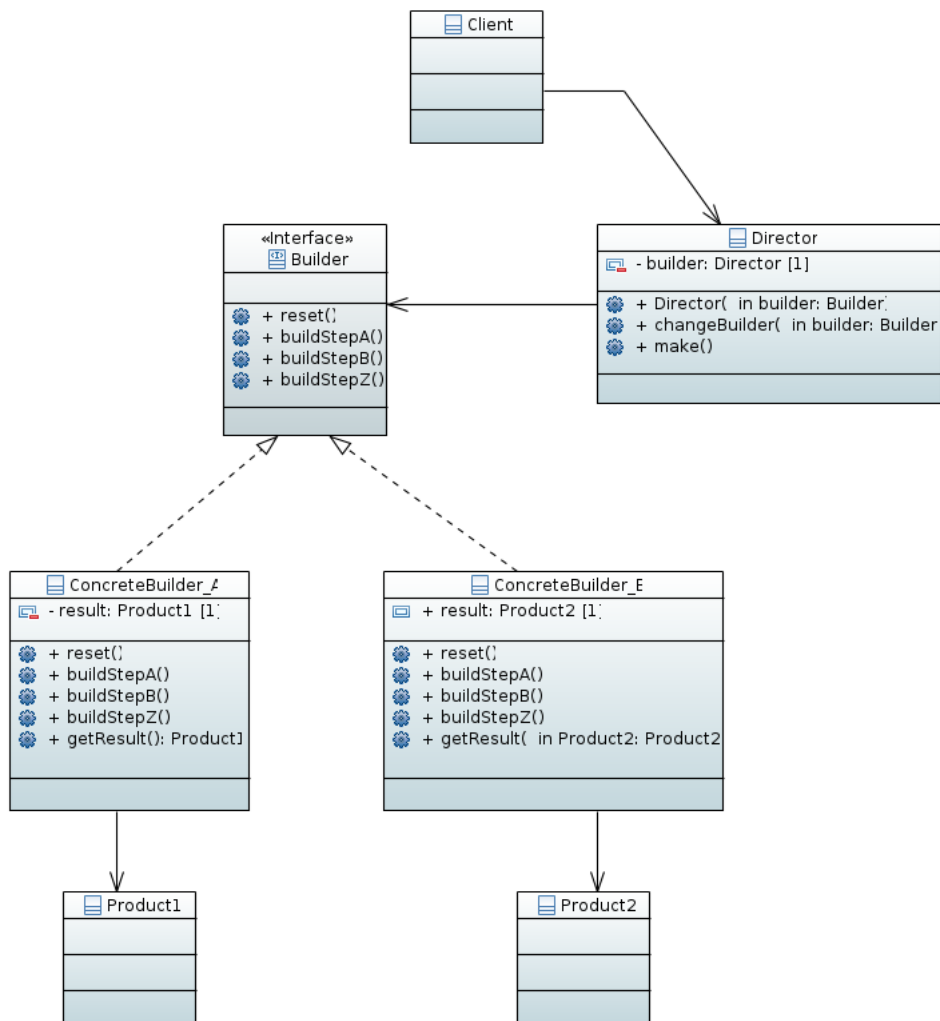


Figure 3: *Builder Class Diagram*

- **Builder** interface declares the production steps common to all the builders.
- **Concrete Builders** implement the construction steps needed for products which do not follow the common interface.
- **Products** are the resulting objects constructed by different builders.
- **Director** might be useful to call the steps needed
- **Client** associates a builder with a director in order to let latter be used for any further construction.

Pros and Cons

Relation with other Design Patterns

3.1.4 Prototype

Prototype enables the copy of existing objects without implementing code which depends on their classes. This is achieved delegating the cloning process to the actual objects that are being

cloned. A common interface is implemented for all the objects that support cloning. The clone method is similar for all classes: it creates an object of the current class and carries over all the fields values of the old cloned one. Copying private fields is possible for any programming language which allows to access private fields of other objects of the same class. An object which could be cloned is called Prototype. Cloning is an alternative to subclassing for objects with many fields and configurations.

When to use this pattern

-
-
-

Structure

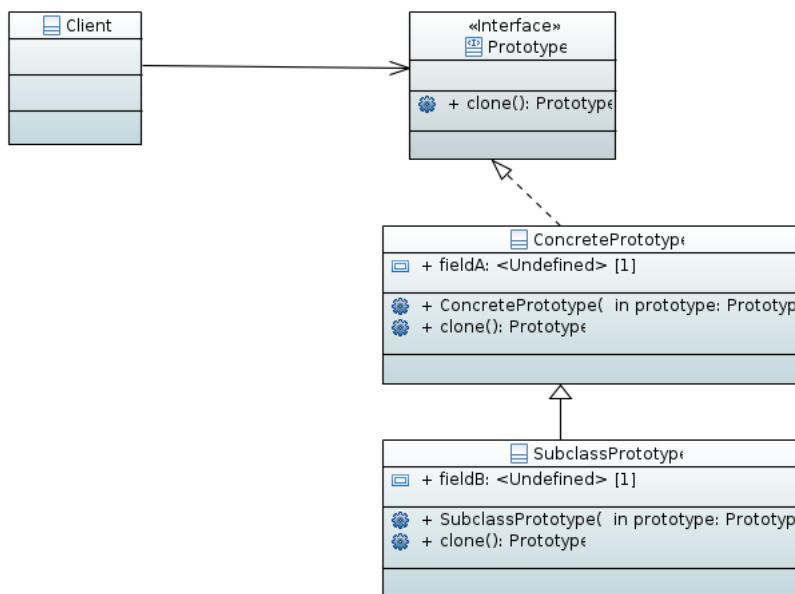


Figure 4: Prototype Class Diagram

- **Prototype** interface declares the cloning method
- **Concrete Prototype** implements the cloning method and may also handle some logic needed for edge cases of the cloning process.
- **Client** can produce a copy of any object which follows the prototype interface.

Pros and Cons

Relation with other Design Patterns

3.1.5 Singleton

Singleton ensures that a class could have one instance only, while providing a global access point to this instance. This functionality could be interpreted as a violation of the Single Responsibility Principle.

When to use this pattern

-
-
-

Structure

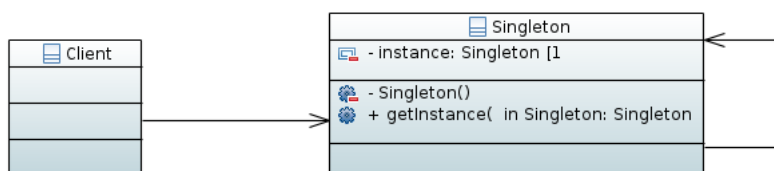


Figure 5: *Singleton Class Diagram*

- **Singleton** class should have a private constructor, in order to prevent the client from calling it. A static method `getInstance` is responsible to create and store the first and only instance in a static field. Successive calls return the same instance.

Pros and Cons

Relation with other Design Patterns

3.2 Behavioral Patterns

3.2.1 Chain of Responsibility

Chain of Responsibility makes possible to pass a request along a chain of handlers, each of which decides upon receiving a request either to process the request or to pass it to the next handler in the chain.

When to use this pattern

-
-
-

Structure

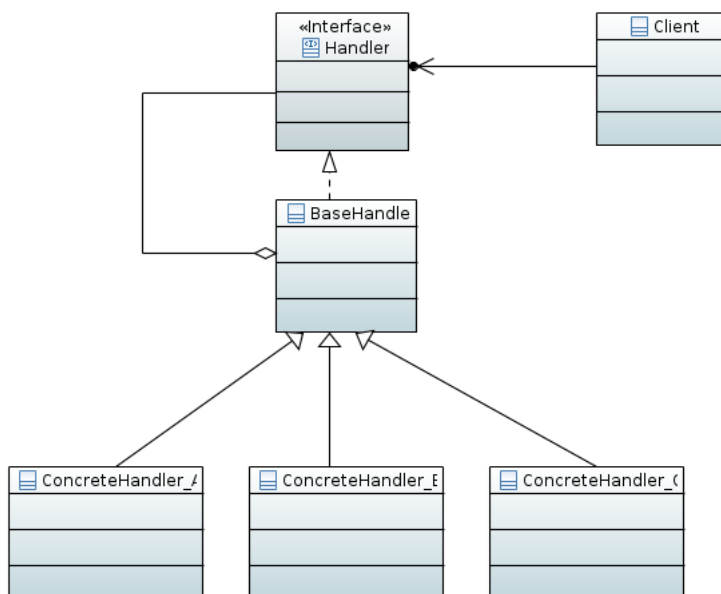


Figure 6: *Chain of Responsibility Class Diagram*

- **Handler** declares a common interface for all the concrete handlers, usually in terms of a first method for handling the requests and second one to set the next handler in the chain.
- **Base Handler** could be optionally implemented
- **Concrete Handlers** contain the actual code to process the request, which will be executed after an optional preliminary check to understand whether it could be processed or passed on.
- **Client** may create chains both statically and dynamically, and depending on its own logic as well. The request can be sent to any handler in the chain.

Pros and Cons

Relation with other Design Patterns

3.2.2 Command

Command

When to use this pattern

-
-
-

Structure

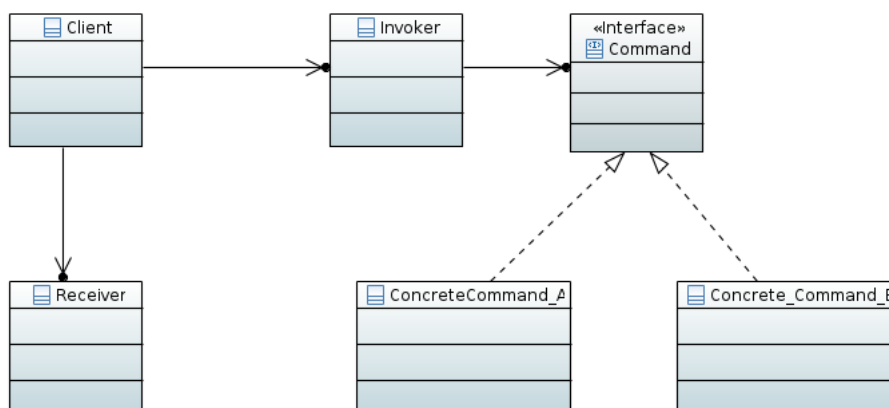


Figure 7: *Command Class Diagram*

- **Sender or Invoker** is responsible for creating and sending requests. For this purpose, the class needs a field to store a reference to a command object. Instead of sending the request directly to the receiver, the sender directly triggers a command objects, which are usually provided and not created by the sender itself.
- **Command** interface declares the method to execute the command.
- **Concrete Command** is meant to implement various kind of requests, passing the call to one of the business logic objects. It is possible to store the parameters needed to execute the command in a field of the class.
- **Receiver** implements some business logic and most of the time does the actual work, while commands only handle the details of how a request is passed to the receiver.
- **Client** creates and configures commands, passing all the parameters needed and a reference to the Receiver.

Pros and Cons

Relation with other Design Patterns

3.2.3 Iterator

Iterator enables the client to traverse elements of a collection without exposing its underlying representation. This is achieved extracting the traversal behavior of a collection into a separate object, called iterator. Details of the traversals are encapsulated into the iterator class; in this way several iterators can go through the same collection at the same time and independently of each other. Iterators should implement the same interface in order to make the client code compatible with any collection type or any traversal algorithm, as long as there's a proper iterator.

When to use this pattern

-
-
-

Structure

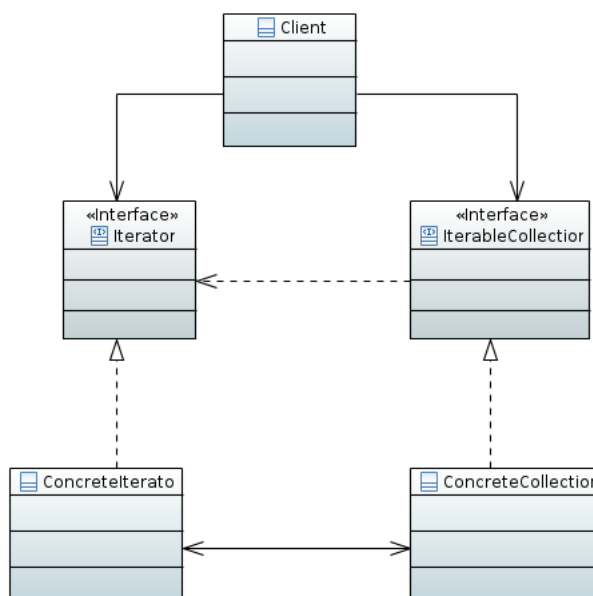


Figure 8: *Iterator Class Diagram*

- **Iterator** interface declares the operations needed to traverse a collection, like fetching the next element, keeping track of the current position and so on.
- **Concrete Iterator** implements specific algorithms each independently on others in order to be able to use many of them at the same time.

- **Collection** interface declares one or multiple methods for getting iterator compatible with the collection. The return type of these methods must match the Iterator interface.
- **Concrete Collection** returns instances of a particular concrete iterator class on client's request.
- **Client** can work with both the collection and the iterator using their interfaces.

Pros and Cons

Relation with other Design Patterns

3.2.4 Mediator

Mediator reduces chaotic and confusing dependencies between objects, restricting direct communications between the objects and forcing them to collaborate only through a mediator object. This is achieved cutting all direct communication between the components which are required to be independent of each other and ...

When to use this pattern

-
-
-

Structure

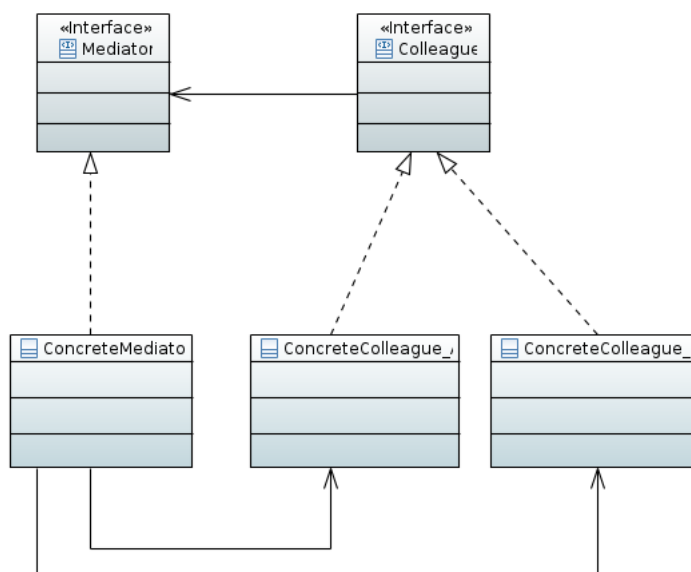


Figure 9: Mediator Class Diagram

- **Components** are classes containing some business logic. Each one has a reference to a mediator, whose referenced type is the Mediator interface's. Components are unaware of the actual Mediator class, so that it could be reused in other contexts with different mediators.
- **Mediator** interface declares the communication structure between components. Components may pass any context as argument of their methods, although they must avoid coupling the receiving component with the sender.
- **Concrete Mediator** implement relation between various components. It could be useful to keep references to all the components managed and their lifecycle as well.

Components are not aware of each other. In case some communication needs to go from a Component to a different one, it must pass through the Mediator. Likewise, the sender doesn't know who will handle the request as well as the receiver doesn't know who sent the request.

Pros and Cons

Relation with other Design Patterns

3.2.5 Memento

Memento

When to use this pattern

-
-
-

Structure

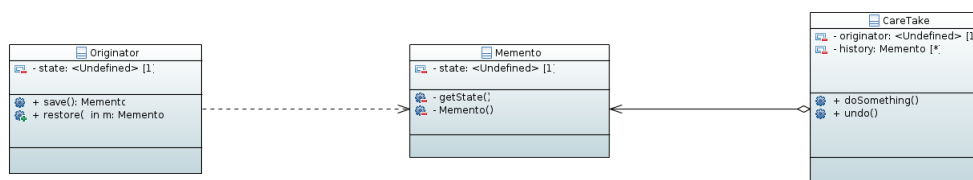


Figure 10: *Memento Class Diagram*

- **Originator** is a generic class which could produce snapshots of its own state, as well as restore its state from a snapshot already available.
- **Memento** is a class which acts like a snapshot. It could be made immutable and pass the data only via the constructor and once.

- **Caretaker** knows the "why" and "when" to capture a snapshot or restore a previous one. This class could manage an history of Originator's' states in a stack data structure.

In the previous design, the Memento class is embedded in the Originator, in order to make it easier to the latter to access the former's methods and data members.

Pros and Cons

Relation with other Design Patterns

3.2.6 Observer

Observer

When to use this pattern

-
-
-

Structure

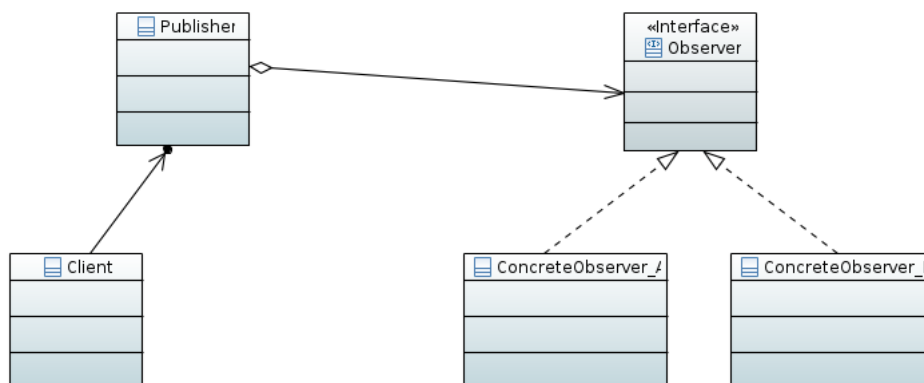


Figure 11: *Observer Class Diagram*

- **Publisher** is a class which issues events of interests to other objects. This class contains a subscription infrastructure that lets subscribers both join and leave the subscriber list.
- **Subscriber** interface declares the notification interface, which in most cases is a single update method with some contextual parameters.
- **Concrete Subscribers** implement the different logics in response to notifications issued by the publisher. As a contextual parameter, a Concrete Publisher
- **Client** creates both publisher and subscriber objects and manage the registrations for updates.

Pros and Cons

Relation with other Design Patterns

3.2.7 State

State

When to use this pattern

-
-
-

Structure

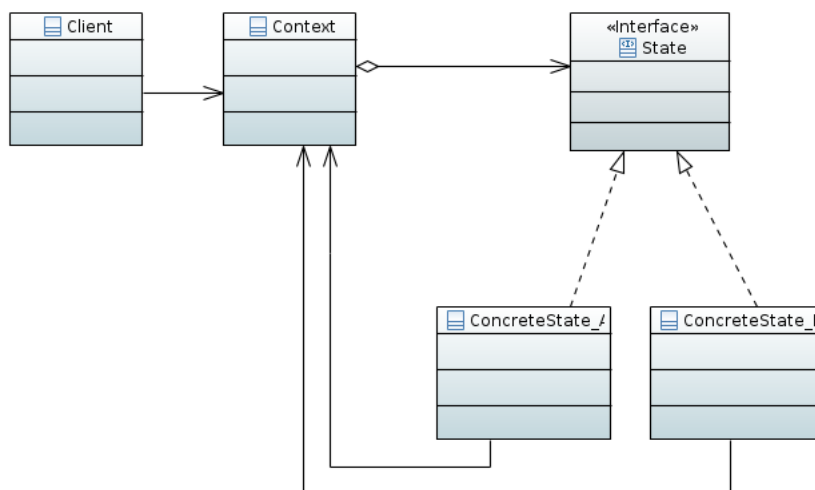


Figure 12: *State Class Diagram*

- **Context** stores a reference to one of the concrete state objects, to which it delegates all the state-related work. The Context interacts with the state interface, in order to be able to interact with any Concrete State.
- **State** interface declares the state generic methods, which should make sense for all concrete states for a matter of consistency.
- **Concrete States** implement their own specific methods. Intermediate abstract classes might help encapsulating some common behavior in order not to duplicate code.

A different Concrete State can be set by both the concrete states themselves.

Pros and Cons

Relation with other Design Patterns

3.2.8 Strategy

Strategy

When to use this pattern

-
-
-

Structure

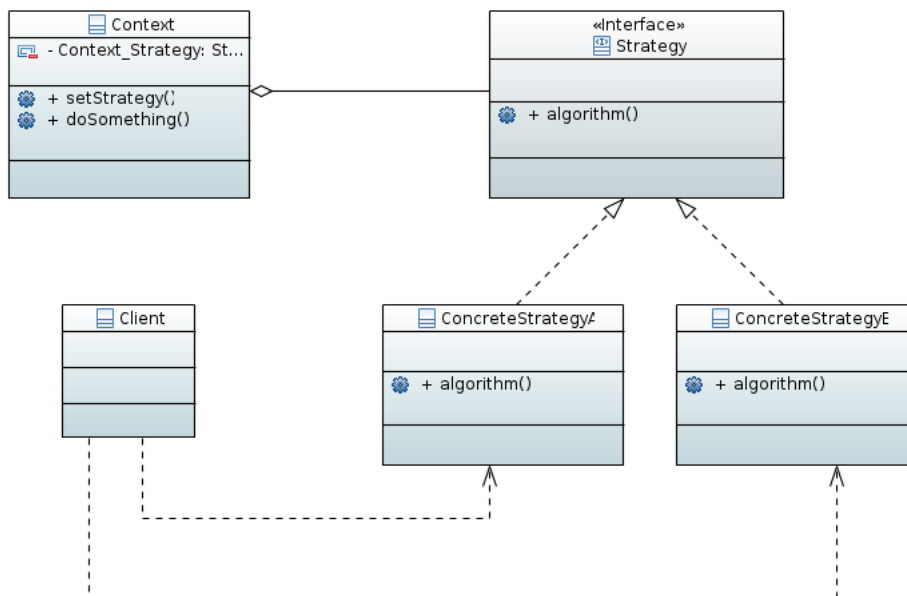


Figure 13: Strategy Class Diagram

- **Context** has a reference to the Strategy Interface, used to reference the Concrete Strategy in use, and methods to replace the strategy at run-time.
- **Strategy** interface define the part which is common to all concrete strategies and declares a method used by the Context to execute it.
- **Concrete Strategies** implement a variety of algorithms needed by the context.
- **Client** manages the creation of the strategies and pass them to the context.

Pros and Cons

Relation with other Design Patterns

3.2.9 Template Method

Template Method

When to use this pattern

-
-
-

Structure

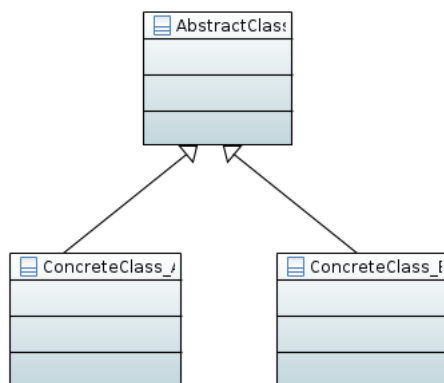


Figure 14: *Template Method Class Diagram*

- **Abstract Class** declares the steps of an algorithm and the **templateMethod** used to call them in a specific order. The steps may both be declared as abstract or implement some actual routines.
- **Concrete Classes** override all of the steps, but not the **templateMethod** itself.

Pros and Cons

Relation with other Design Patterns

3.2.10 Visitor

Visitor

When to use this pattern

-
-
-

Structure

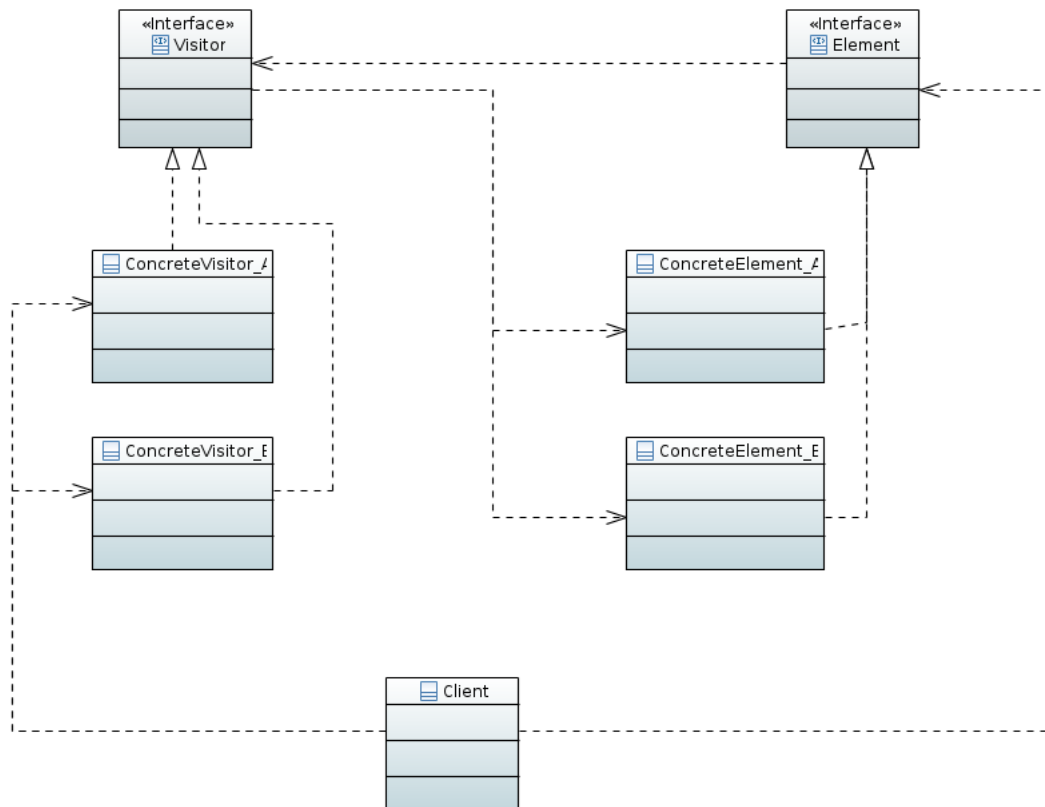


Figure 15: Visitor Class Diagram

- **Visitor** interface declares a set of visiting methods, each with a different Concrete Element taken as argument.
- **Concrete Visitor** implements the different behaviors for each Concrete Element.
- **Element** interface declares a method to accept the Visitor interface as argument.
- **Concrete Element** implements the acceptance method, whose purpose is the redirection the call to the proper visitor's method for the current element class. This method must be overridden in any subclass even though it was already developed in the base one.
- **Client** usually needs to work on a collection or a similar complex object whose type is ignored by the Client, since it works with the abstract interface.

Pros and Cons

Relation with other Design Patterns

3.3 Structural Patterns

3.3.1 Adapter

Adapter allows the collaboration among objects with incompatible interfaces. This is achieved through the implementation of an Adapter which make one interface understandable by an object of a different class. The adapter gets an interface compatible with one existing object. Through this interface, the objects can call the adapter's methods. Upon receiving a call, the adapter passes the request to the second object, in a format and order it could understand. An Adapter could work in both directions.

When to use this pattern

-
-
-

Structure

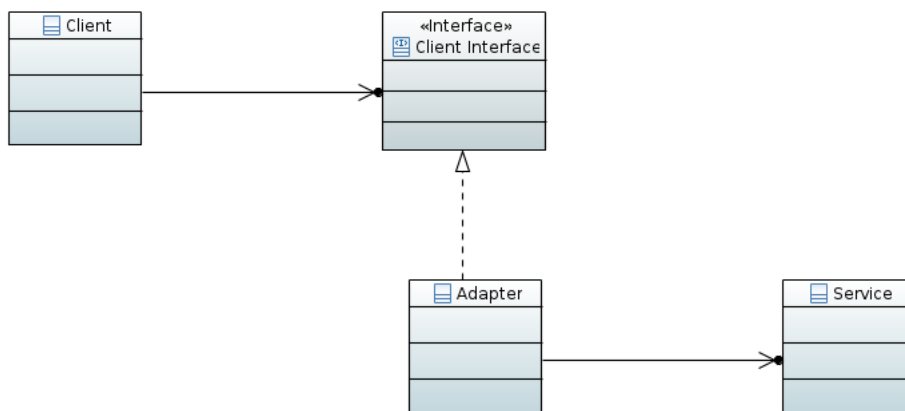


Figure 16: *Adapter Class Diagram*

- **Client** is the business logic of the program.
- **Client Interface** defines the protocol for a class to be able to collaborate with it.
- **Service** is usually a 3rd party or legacy software with an interface incompatible with the Client.
- **Adapter** is a class able to work with both the client and the service. It receives calls from the client and translates into calls to a wrapped service object.

Since the Client interacts with the adapter through the client interface, it doesn't get coupled. Therefore, different adapters can be added to the program without breaking any existing code.

Pros and Cons

Relation with other Design Patterns

3.3.2 Bridge

Bridge splits a large class or a set of closely related classes into two separate hierarchies which can be developed independently of each other. This is achieved by switching from inheritance to object composition. One of the dimensions is extracted into a separate class hierarchy, therefore the original classes will reference an object of the new hierarchy instead of having all its states and behaviors coded within the class. The original class implement a reference field for each "extracted-dimension" classes This extraction makes a class able to delegate any 'dimension-related' work to the linked 'dimension-related' object.

When to use this pattern

-
-
-

Structure

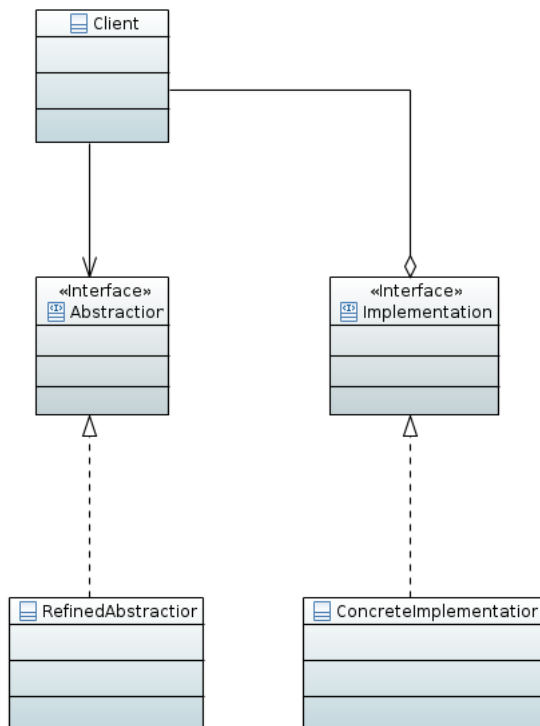


Figure 17: Bridge Class Diagram

- **Abstraction** provides high-level interface, and relies on the Implementation objects to perform the actual work.
- **Implementation** is the interface shared by all the concrete implementations. Usually the abstraction declares some complex behavior that relies on primitive operations declared by the implementation.
- **Concrete Implementations** implement specific code.
- **Refined Abstraction** are optional and might help providing variants of control logic.
- **Client** interacts with the abstraction, after linking it with one of the implementation objects.

Pros and Cons

Relation with other Design Patterns

3.3.3 Composite

Composite after enabling composition of objects into tree structures, this pattern let clients work with these structures as they were individual objects. This is achieved extending the class

When to use this pattern

-
-
-

Structure

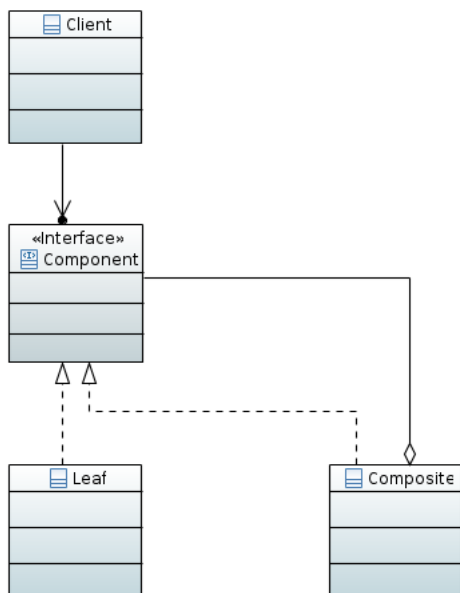


Figure 18: *Composite Class Diagram*

- **Component** interface contains the common operations for both simple and complex elements of the tree.
- **Leaf** is a basic element of a tree and doesn't contain any other sub-element. Usually is the part of the system which ends up doing the work.
- **Container** or Composite is an element that has sub-elements, which means leafs and containers as well. It interacts with sub-elements through their interfaces.
- **Client** can work with both simple and complex elements through the component interface.

Pros and Cons

Relation with other Design Patterns

3.3.4 Decorator

Decorator attaches new behaviors to objects by placing these objects inside special wrapper objects containing the desired behavior.

When to use this pattern

-
-
-

Structure

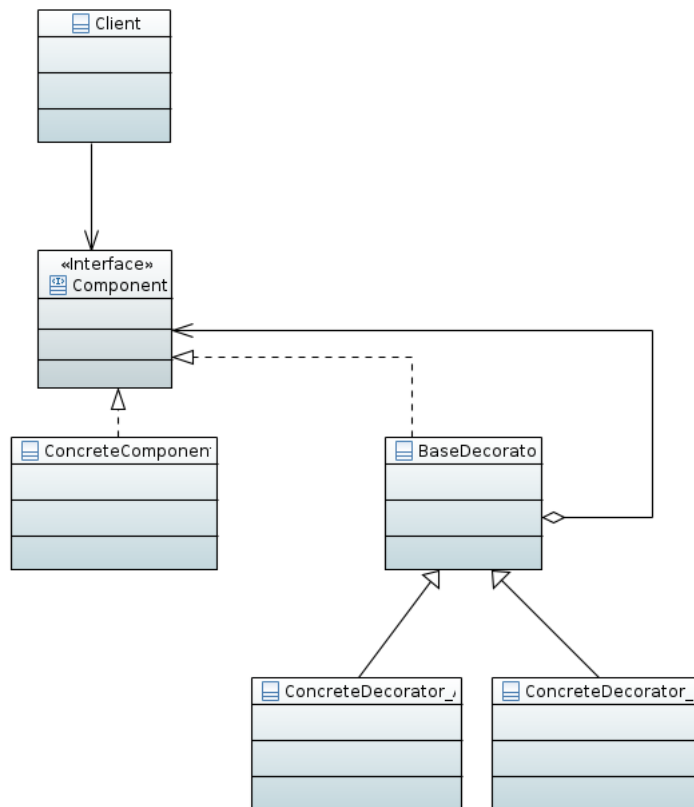


Figure 19: *Decorator Class Diagram*

- **Component** is the interface common for both wrapper and wrapped objects.
- **Concrete Component** is a class being wrapped and whose basic behavior can be altered by decorators.
- **Base Decorator** has a reference of the wrapped object, whose referenced type matches the Component interface. The Base Decorator delegates all operations to the wrapped object.
- **Concrete Decorators** define behaviors which can be added to Components dynamically. The concrete decorator overrides the base behavior and execute the variant either before or after calling the parent method.

Multiple layers of decorations are possible as long as the Client interacts with the object through the component.

Pros and Cons

Relation with other Design Patterns

3.3.5 Facade

Facade consists of a simplified interface to a complex subsystem which contains lots of moving parts. Functionalities provided by the Facade could be reduced to the subset the client is interested in.

When to use this pattern

-
-
-

Structure

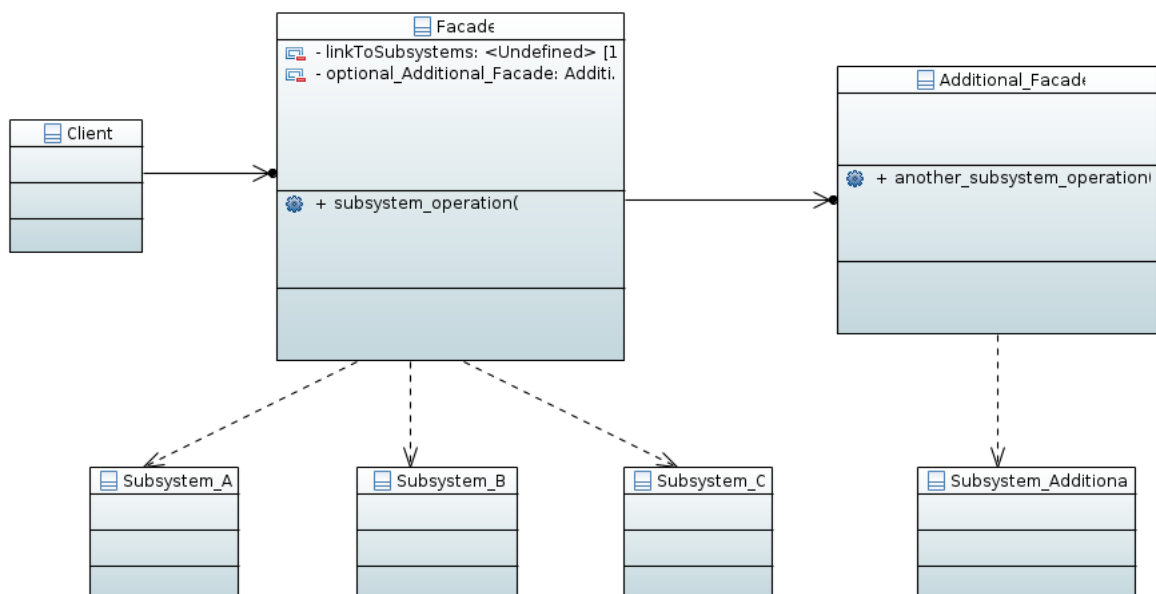


Figure 20: *Facade Class Diagram*

- **Facade** implements a convenient access to a particular part of the subsystem's functionality and knows where to direct the client's request and how to operate all the system's parts.
- **Additional Facade** might help reducing the tasks managed by a single Facade. It can be used by the Client and by the other facades as well.
- **Complex Subsystem** Subsystem class are various object which need to be studied deeply in order to manage them properly. These classes work together and are unaware of the Facade's existence.
- **Client** calls the subsystem classes through the Facade only.

Pros and Cons

Relation with other Design Patterns

3.3.6 Flyweight

Flyweight reduces RAM consumption by sharing common parts of state between multiple objects, instead of creating copies of same data in each object.

When to use this pattern

-
-
-

Structure

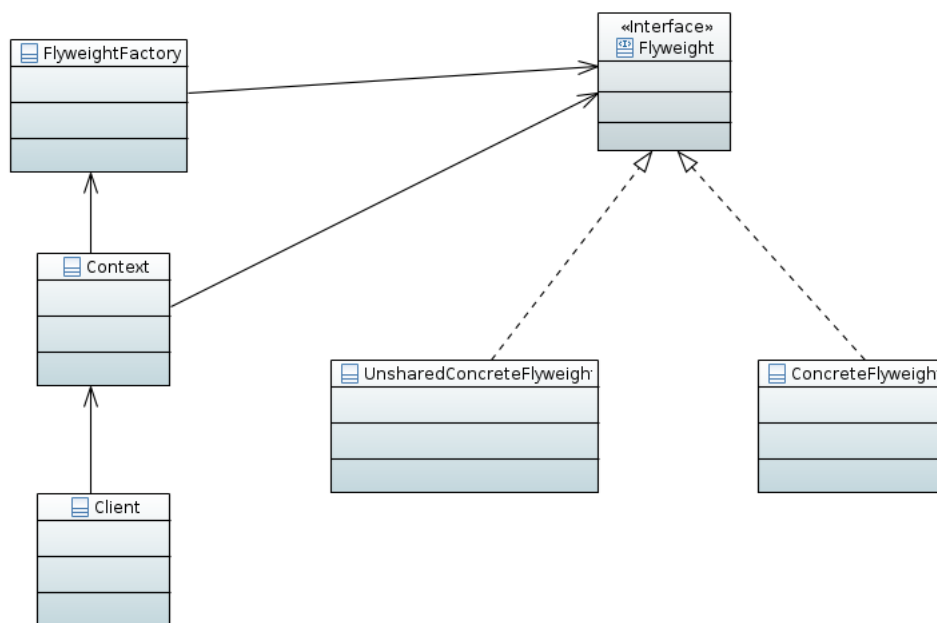


Figure 21: *Flyweight Class Diagram*

- **Flyweight** contains the portion of the original object's state that can be shared between multiple objects. The state stored inside a flyweight is called *intrinsic*; the state passed to the flyweight's methods is called *extrinsic*. The same flyweight object can be used in many contexts.
- **Context** contains the extrinsic state, unique across all original objects. It represents the full state when paired with one of the flyweight objects.

- **Flyweight Factory** manages a pool of existing flyweights freeing the Client from the task of creating. The Flyweight Factory receives a call to retrieve a specific flyweight, looks over previously created flyweight and either returns one of these or creates a new one in case nothing matches the search criteria.
- **Client** sees the flyweight as a template object which can be configured at runtime by passing some contextual data into parameters of its methods.

Pros and Cons

Relation with other Design Patterns

3.3.7 Proxy

Proxy

When to use this pattern

-
-
-

Structure

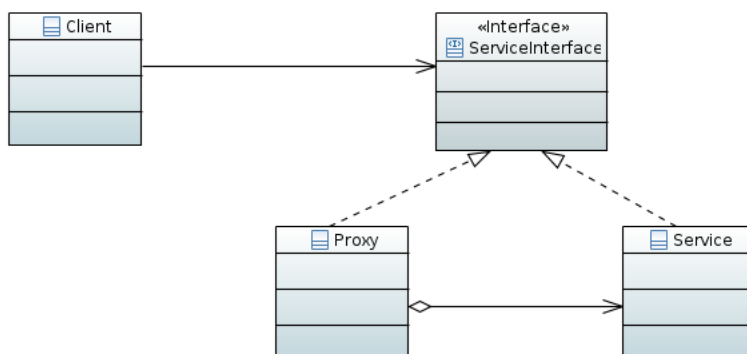


Figure 22: Proxy Class Diagram

- **Service Interface** declares the interface to the Service, which must be followed by the Proxy in order to be able to disguise itself as a service object.
- **Service** is a class whose business logic is proxied.
- **Proxy** references a service object and manages to process requests and pass them on to the service object, whose lifecycle sometimes could be managed by the Proxy as well.
- **Client** can work with a Proxy until the latter's interface is the same as the Service's.

Pros and Cons

Relation with other Design Patterns

4

SECTION

Idioms

4.1 PImpl Idiom

[cppreference documentation](#) about the PImpl Idiom.

"Pointer to implementation" or "pImpl" is a C++ programming technique that **removes implementation details of a class from its object representation** by placing them in a separate class, accessed **through an opaque pointer**.

This idiom is also known as **Compiler Firewall Idiom**.

Problematic Scenario

Whenever file class definition changes in a header, all users of that class must be recompiled. The reason is the textual inclusion on which the C++ build model is based. The user is assumed to know two features which could be affected by private members: **size and layout** and **functions**.

Solution

```
1 // Pimpl idiom - basic idea
2 class widget {
3     // :::
4 private:
5     struct impl;           // things to be hidden go here
6     impl* pimpl_;         // opaque pointer to forward-declared class
7 };
```

4.2 Fast PImpl Idiom

4.3 Handle/Object Idiom

4.4 Copy and Swap Idiom

This idiom aims at creating a strong exception-safe implementation of the overloaded assignment operator.

This idiom makes use of the **RAII** idiom in order to acquire the new resource before it forfeits its current resource. If the acquisition is successful

4.5 RAII - Resource acquisition is initialization

5

SECTION

Containers

The [Containers library](#) is a generic collection of class templates and algorithms that allow programmers to easily implement common data structures like queues, lists and stacks.

The container classes are:

- sequence containers
- associative containers, which can be searched in $O(\log n)$.
- unordered associative containers, which can be searched in $O(1)$ on average and in $O(n)$ in the worst one.

5.1 Links to the CPP Reference documentation

5.1.1 Sequence Containers

[array](#) | [vector](#) | [deque](#) | [forward_list](#) | [list](#)

5.1.2 Associative Containers

[set](#) | [map](#) | [multiset](#) | [multimap](#)

5.1.3 Unordered Associative Containers

[unordered_set](#) | [unordered_map](#) | [unordered_multiset](#) | [unordered_multimap](#)

5.1.4 Container Adaptors

[stack](#) | [queue](#) | [priority_queue](#) | [flat_set](#) | [flat_map](#) | [flat_multiset](#) | [flat_multimap](#)

5.2 Invalidation of Iterators and References

Category	Container	Valid after insertion		Valid after erasure		Conditionality
		iterators	reference	iterators	reference	
Sequence Containers	array	N/A		N/A		
	vector	No		N/A		Insertion changed capacity
		Yes		Yes		Before modified element(s) (for insertion only if capacity didn't change)
		No		No		At or after modified element(s)
	deque	No	Yes	Yes, but erased one(s)		Modified first or last element
			No	No		Modified middle only
	list	Yes		Yes, but erased one(s)		
	forward_list	Yes		Yes, but erased one(s)		
Associative Containers	set multiset map multimap	Yes		Yes, but erased one(s)		
Unordered associative Containers	unordered_set unordered_multiset	No	Yes	N/A		Insertion caused rehash
	unordered_map unordered_multimap	Yes		Yes, but erased one(s)		No rehash

Memory Management and Smart Pointers

6.1 Memory Management

Allocation means to assign, allot, distribute or "set apart for a particular purpose". Programs manage their memory partitioning or dividing it into different units performing specific tasks.

In C++, there are two ways to create objects and this aspect reflects on the existence of two different units used to allocate the memory, the **stack** and **heap**, which manage the program's used/unused memory in two different ways.

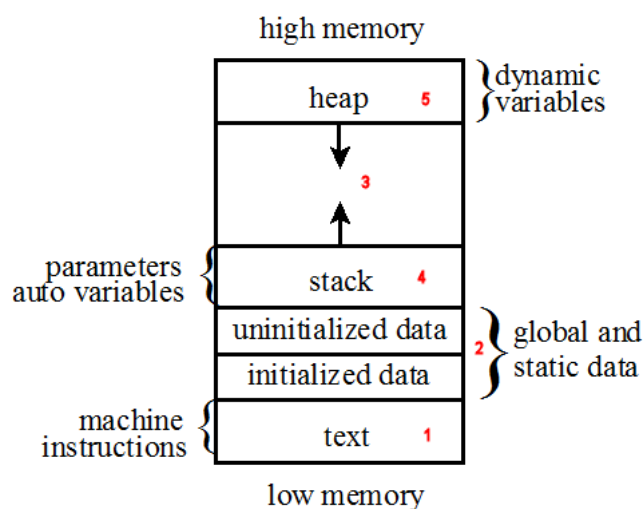


Figure 23: Functional memory organization of a running program

Text area contains machine instructions (i.e. the executable code). **Global** and **static**

variables are typically allocated at program startup. **Stack** contains all the automatic variables. **Heap** is meant to allocate the memory for the dynamic variables through the **new** and **delete** operators. The space in between is allocated by the operating system but not used, and is meant to let both the heap and stack grow.

Stack

This is a simple LIFO (last in, first out) data structure which must support at least the **push** and **pop** operations. Even though a stack only allows accesses at the top, it is easy to implement and fast to execute push and pop operations.

Heap

This is a more flexible data structure, which allows programs to allocate or deallocate memory anywhere and in any convenient order. This might cause hole in the heap, i.e. unallocated memory surrounded by allocated memory.

The only restriction for the heap is that memory can be allocated in a contiguous block of memory large enough to satisfy the request with a single chunk of memory. This restriction has two effects: it requires the operating system to scan the heap until a contiguous block is found, and to coalesce adjacent blocks of memory returned by the program.

Side effect is that the operation on the heap are slower than the operations on the stack.

6.2 Smart Pointers in C++

[Smart pointers](#) enable automatic, exception-safe, object lifetime management.

Generally speaking, a **smart pointer** is a data type whose value can be used like a pointer, but which provides the additional feature of memory management which frees the memory when the smart pointer is no longer used. A smart pointer should therefore be used any time the ownership of memory needs to be tracked.

6.2.1 `std::unique_ptr`

[cppreference documentation](#) about `std::unique_ptr`.

`std::unique_ptr` is a smart pointer that **owns and manages** another object through a pointer and **disposes of** that object when the `std::unique_ptr` goes out of scope.

The object is disposed of, using the associated deleter when either of the following happens:

- the managing `std::unique_ptr` object is destroyed
- the managing `std::unique_ptr` object is assigned another pointer via `operator=` or `reset()`.

Notes

Only **non-const** `std::unique_ptr` can **transfer the ownership** of the managed object to another `std::unique_ptr`.

If an object's lifetime is managed by a `const std::unique_ptr`, it is limited to the scope in which the pointer was created.

`std::unique_ptr` is commonly used to manage the lifetime of objects, including:

- providing **exception safety** to classes and functions that handle objects with dynamic lifetime, by **guaranteeing deletion** on both normal exit and exit through exception
- **passing ownership** of uniquely-owned objects with dynamic lifetime into functions
- **acquiring ownership** of uniquely-owned objects with dynamic lifetime from functions
- as the element type in move-aware containers, such as `std::vector`, which hold pointers to dynamically-allocated objects (e.g. if **polymorphic behavior** is desired).

`std::unique_ptr` may be constructed for an **incomplete type** `T`, such as to facilitate the use as a handle in the `pImpl` idiom.

Member types

pointer	Must satisfy NullablePtr
element_type	<code>T</code> , the type of the object managed by this <code>std::unique_ptr</code>
deleter_type	Deleter, the function object or lvalue reference to function or to function object, to be called from the destructor

Member functions

constructor	constructs a new <code>unique_ptr</code>
destructor	destructs the managed object if such is present
operator=	assigns the <code>unique_ptr</code>

Modifiers

release	returns a pointer to the managed object
reset	returns the deleter that is used for destruction of the managed object
swap	checks if there is an associated managed object

Observers

<code>get</code>	returns a pointer to the managed object
<code>get_deleter</code>	returns the deleter that is used for destruction of the managed object
<code>operator bool</code>	checks if there is an associated managed object

Single-object version, `unique_ptr`

<code>operator*</code> <code>operator-></code>	dereferences pointer to the managed object
--	--

6.2.2 `std::shared_ptr`

[cppreference documentation](#) about `std::shared_ptr`.

This is a smart pointer that retains shared ownership of an object through a pointer.

Several `shared_ptr` objects may own the same object.

The object is destroyed and its memory deallocated when either of the following happens:

- the last remaining `shared_ptr` owning the object is destroyed;
- the last remaining `shared_ptr` owning the object is assigned another pointer via `operator=` or `reset()`.

A `std::shared_ptr` can share ownership of an object while storing a pointer to another object.

A `std::shared_ptr` may also own no objects, in which case it is called empty (an empty `std::shared_ptr` may have a non-null stored pointer if the aliasing constructor was used to create it).

Member functions

constructor	constructs a new <code>unique_ptr</code>
destructor	destructs the managed object if such is present
operator=	assigns the <code>unique_ptr</code>

Modifiers

reset	replaces the managed object
swap	swaps the managed objects

Observers

get	returns the stored pointer
operator* operator->	dereferences the stored pointer
operator[]	provides indexed access to the stored array
use_count	returns the number of <code>shared_ptr</code> objects referring to the same managed object
unique	checks if the managed object is managed only by the current <code>shared_ptr</code> instance
operator bool	checks if the stored pointer is not null
owner_before	provides owner-based ordering of shared pointers

6.2.3 `std::weak_ptr`

[cppreference documentation](#) about `std::weak_ptr`.

`std::weak_ptr` is a smart pointer that holds a non-owning ("weak") reference to an object that is managed by `std::shared_ptr`. It must be converted to `std::shared_ptr` in order to access the referenced object.

`std::weak_ptr` models temporary ownership

Member functions

constructor	creates a new <code>weak_ptr</code>
destructor	destroys a <code>weak_ptr</code>
operator=	assigns a <code>weak_ptr</code>

Modifiers

reset	releases the ownership of the managed object
swap	swaps the managed objects

Observers

use_count	returns the number of <code>shared_ptr</code> objects that manage the object
expired	checks whether the referenced object was already deleted
lock	creates a <code>shared_ptr</code> that manages the referenced object
owner_before	provides owner-based ordering of weak pointers

6.2.4 `std::auto_ptr`

[cppreference documentation](#) about `std::auto_ptr`.

It was deprecated in C++11 and removed in C++17.

7.1 C++ Good Practices

7.1.1 Rule of three/five/zero

Rule of three

If a class requires a **user-defined destructor**, a user-defined **copy constructor**, or a user-defined **copy assignment operator**, it almost certainly requires all three.

Because C++ copies and copy-assigns objects of user-defined types in various situations (passing/returning by value, manipulating a container, etc), these special member functions will be called, if accessible, and if they are not user-defined, they are implicitly-defined by the compiler.

The implicitly-defined special member functions are typically incorrect if the class manages a resource whose handle is an object of non-class type (raw pointer, POSIX file descriptor, etc), whose destructor does nothing and copy constructor/assignment operator performs a "shallow copy" (copy the value of the handle, without duplicating the underlying resource).

Classes that manage non-copyable resources through copyable handles may have to declare copy assignment and copy constructor private and not provide their definitions or define them as deleted. This is another application of the rule of three: deleting one and leaving the other to be implicitly-defined will most likely result in errors.

Rule of five

Because the presence of a user-defined destructor, copy-constructor, or copy-assignment operator prevents the implicit definition of the move constructor and the move assign-

ment operator, **any class for which move semantics are desirable**, has to declare all five special member functions.

Unlike Rule of Three, failing to provide move constructor and move assignment is usually not an error, but a missed optimization opportunity.

Rule of zero

Classes that have custom destructors, copy/move constructors or copy/move assignment operators should deal exclusively with ownership (which follows from the Single Responsibility Principle). Other classes should not have custom destructors, copy/move constructors or copy/move assignment operators. (see [?])

This rule also appears in the C++ Core Guidelines as C.20: If you can avoid defining default operations, do.

When a **base class is intended for polymorphic use**, its destructor may have to be declared public and virtual. This blocks implicit moves (and deprecates implicit copies), and so the special member functions have to be declared as defaulted (see [?])

However, this makes the class prone to slicing, which is why polymorphic classes often define copy as deleted (see [?]), which leads to the following generic wording for the Rule of Five: If you define or =delete any default operation, define or =delete them all (see [?])

7.1.2 Const Correctness

Const Correctness consists in using the keyword `const` to prevent `const` objects from being mutated.

const and pointer combinations

Reading it right-to-left

<code>const X* p</code>	p is a pointer to an X that is constant.
<code>X const* p</code>	equivalent to <code>const X* p</code> .
<code>X* const p</code>	p is a const pointer to an X that is not const.
<code>const X* const p</code>	p is a const pointer to an X that is const.

The second combination is equivalent to the first one because of the **const-on-the-right style**, which require to write the `const` specifier on the right of the object it makes `const`. This style is also called "consistent const" since it is consistent with the way of defining a `const` member function. Moreover, it is also more consistent with pointer declarations.

const and reference combinations

Reading it right-to-left The equivalence between the second and the first combination is due to the same reason valid for the pointers.

The last two combinations are nonsense and should be avoided. They are redundant as a reference is always constant, in the sense that you can never resear a reference to make it refer to a different object.

<code>const X& r</code>	<code>r</code> aliases an X object, which can not be changed via <code>r</code>
<code>X const& r</code>	equivalent to <code>const X& r</code>
<code>X& const r</code>	nonsense functionally equivalent to <code>X& r</code>
<code>const X& const r</code>	nonsense functionally equivalent to <code>const X& r</code>

Logical and Physical state

When reasoning in terms of const-correctness, it's important to distinguish the logical and the physical state of an object.

On the inside, an object has a **physical state**, also called **concrete** or **bitwise** state. This state is the one visible to the programmer who implements the software. From the outside, the client has an access to the object restricted to public member functions and friend functions. What is visible is the **logical** or **abstract** or **meaningwise state**.

A lookup functionality for a collection-object is a good example to show how there could be bits in the object's physical state that have no corresponding elements in the object's logical state. The lookup functionality might improve the performance using a cache to store the already looked-up items. The implementation of the cache is part of the object's physical state but the implementation is internal and the details will probably not be exposed to the users, i.e. it is not part of the object's logical state.

constness of public member functions

The constness of a public method must make sense to the object's users, and those users can see only the object's logical state. Therefore, in this context the constness to take care of is the logical one.

- If a method changes any part of the object's logical state, it logically is a mutator; it should not be const even if (as actually happens!) the method doesn't change any physical bits of the object's concrete state.
- Conversely, a method is logically an inspector and should be const if it never changes any part of the object's logical state, even if (as actually happens!) the method changes physical bits of the object's concrete state.

return-by-reference and const member function

A const member function must not change nor allow its caller to change the logical state of the `this` object it's been called on.

const overloading

mutable keyword

Error converting F** into const F**

7.2 C++11 New Features

7.2.1 Value Categories

[cppreference documentation](#) about value categories.

Due to the expressiveness of the C++ language, the terms **lvalue** and **rvalue** references evolved in a more complex set.

glvalues

Generalized lvalue is used to refer to something that could be either a **lvalue** or **xvalue**.

rvalues

This term is inherited from C, in which **rvalues** can be on the right side of an assignment. In C++ this term can refer to things that are either **xvalues** and **prvalues**.

lvalues

This term is also inherited from C, in which **lvalues** can be on the left side of an assignment. Therefore, the simplest one is the name of a variable in a declaration.

Any expression resulting in an **lvalue reference** is also an **lvalue**.

```
1 int v;           // v is an lvalue of type int
2 A* p1;           // *p1 is an lvalue
3 A& r1 = v1;      // r1 is an lvalue
4 A& f();           // f() is an lvalue
```

Accessing members of objects through pointers or references yields **lvalues**. For example, with the following code

```
1 struct Foo
2 {
3     Bar a;
4     Bar b;
5 };
6
7 Foo& f2();
8 Foo* p2;
9 Foo v2;
```

`f2().a`, `p2->b` and `v2.a` are all **lvalues** if type A.

String literals are **lvalues** of type array.

To conclude, a **named object** declared with an **rvalue reference** is an **lvalue**. The name rvalue reference indicates that it can bind to an **rvalue**, and once you've declared a variable and given it a name it is an **lvalue**.

```
1 void Foo(A&& bar)
2 {
3     // within Foo, bar is an lvalue
```

```

4     // which will only bind to rvalues
5 }

```

xvalues

This term indicates eXpiring value: an unnamed object that will be destroyed soon.

xvalues can be treated either as **glvalues** or **rvalues**, based on the context.

Usually, **xvalues** only arise through **explicit casts** and **function calls**. If an expression is cast to an rvalue reference to some type **T** then the result is an xvalue of type **T**

```

1 static_cast<A&&>(v1) // yields an xvalue of type A

```

If the return type of a function is an **rvalue reference** to some type **T**, the result is an **xvalue** of type **T**. For example, the `std::move` is declared as

```

1 template <typename T>
2 constexpr remove_reference_t<T>&& move(T&& t) noexcept;

```

`std::move(v1)` is an **xvalue** of type **A**. The type deduction rules deduce **T** to be **A&** because `v1` is an **lvalue**.

prvalues

Pure **rvalue** is an **rvalue** which is not an **xvalue**

Literals other than string literals are **prvalues**. For example, `42` is a *prvalue* of type `int` and `3.141f` is a *prvalue* of type `float`

Temporaries are **prvalues**. For example any temporaries created as a result of an implicit conversion is also a **prvalue**. In the following code

```

1 int consume_string(std::string&& s);
2 int i = consume_string("hello");

```

the string literal in the function call will implicitly convert to a temporary of type `std::string`, which can bind to the rvalue reference used for the function parameter since the temporary is a *prvalue*

Reference Binding

[cppreference documentation](#) about reference declaration.

The main difference between **lvalues** and **rvalues** is the way they bind to references. The differences in type deduction rules can have a big impact too.

lvalue references

These references are declared with a single ampersand `&`.

A **non-const lvalue reference** will only bind to **non-const lvalues** of the same type, or a class derived from the referenced type

```

1 class C : public A{};
2
3 int i = 42;
4 A a;
5 B b;

```

```

6 C c;
7 const A ca{};
8
9 A& r1 = a;
10 A& r2 = c;
11 A& r3 = b; // error, wrong type
12 int& r4 = i;
13 int& r5 = 42; // error, cannot bind rvalue
14 A& r6 = ca; // error, cannot bind const object
15 // to non const ref
16 A& r7 = r1;
17 A& r8 = A(); // error, cannot bind rvalue
18 A& r9 = B().a; // error, cannot bind rvalue
19 A& r10 = C(); // error, cannot bind rvalue

```

A **const lvalue reference** can also bind to **rvalues**. The object bound to the reference must have the same type as the referenced type or a class derived from the referenced type. It is possible to bind both const and non-const values to a const lvalue reference.

```

1 const A& cr1 = a;
2 const A& cr2 = c;
3 const A& cr3 = b; // error, wrong type
4 const int& cr4 = i;
5 const int& cr5 = 42; // rvalue can bind OK
6 const A& cr6 = ca; // OK, can bind const object to const ref
7 const A& cr7 = cr1;
8 const A& cr8 = A(); // OK, can bind rvalue
9 const A& cr9 = B().a; // OK, can bind rvalue
10 const A& cr10 = C(); // OK, can bind rvalue
11 const A& cr11 = r1;

```

Binding a temporary object (i.e. a **prvalue**) to a const lvalue reference **extends the lifetime** of that temporary to the lifetime of the reference. In the previous example, it is possible to use r8, r9 and r10 later in the code.

Lifetime extension does not extend to references initialized from the first reference. If a function parameter is a const lvalue reference and gets bound to a temporary passed to the function call, the temporary is destroyed when the function returns. This happens also if the reference was stored in a longer-lived variable, such as a member of a newly constructed object.

volatile and **const volatile** lvalue references are much less used but behave as expected: volatile T& bind to a volatile or non-volatile, non-const lvalue of type T or a class derived from T. However, volatile const lvalue references do not bind to rvalues.

rvalue references

An **rvalue reference** can be used to extend the lifetime of temporary objects, making the referenced object modifiable through them. The reference must be const in order to bind to a const object, though const rvalue reference are much rarer.

```

1 const A make_const_A();
2 A&& rr1 = a; // error, cannot bind lvalue

```

```

3                                     // to rvalue reference
4 A&&      rr2  = A();
5 A&&      rr3  = B();                // error, wrong type
6 int&&    rr4  = i;                  // error, cannot bind lvalue
7 int&&    rr5  = 42;
8 A&&      rr6  = make_const_A();    // error, cannot bind
9                                     //          const object
10                                     //          to non const ref
11 const A&& rr7  = A();
12 const A&& rr8  = make_const_A();
13 A&&      rr9  = B().a;
14 A&&      rr10 = C();
15 A&&      rr11 = std::move(a);      // std::move returns an rvalue
16 A&&      rr12 = rr11;              // error rvalue references
17                                     //          are lvalues

```

rvalue references extend the lifetime of temporary objects in the same way const lvalue references do, therefore temporaries associated with rr2, rr5, rr7, rr8, rr9 and rr10 remain valid until the corresponding references are destroyed.

Reference Collapsing

In C++ it is possible to form reference to reference in **templates** and typedef expressions. In this context, the reference collapsing rule apply: rvalue reference to rvalue reference collapses to an rvalue reference, all the others collapse to lvalue references.

```

1 typedef int&   lref;
2 typedef int&& rref;
3 int n;
4
5 lref&  r1 = n; // type of r1 is int&
6 lref&& r2 = n; // type of r2 is int&
7 rref&  r3 = n; // type of r3 is int&
8 rref&& r4 = 1; // type of r4 is int&&

```

Implicit conversion

A reference-to-A can be initialized with any D object which has an implicit conversion operator that returns A&. In this case the reference is bound to the result of the conversion operator.

A const lvalue-reference-to-E will bind only to objects implicitly convertible to E. The reference is bound to the temporary resulting from the conversion.

General properties

In general, **rvalues** can't be modified and their address can't be taken.

However, as long as class X has a member function named do_something, the expression X().do_something is valid.

ref qualifiers along with const can be used to tag different overloads of class member functions, to indicate they can be applied to only *lvalues* or to only *rvalues*.

When the type of a variable or function template parameter is deduced from its initializer, whether the initializer is an lvalue or rvalue can affect the deduced type.

7.2.2 Move Semantic

Move semantics is a new way of moving resources around in an optimal way by **avoiding unnecessary copies** of temporary objects. It is based on rvalue references and plays an **important role in exception safety** for C++.

7.2.3 Perfect Forwarding

[cppreference documentation](#) about **forward** utility.

[cppreference documentation](#) about **forwarding references**.

Perfect forwarding is an important C++0x feature built on top of rvalue references. It provides the programmer with the capability to automatically apply the move semantics, even when there are intervening function calls to separate the source and the destination of a move.

Examples include constructors and setter functions, that can forward arguments they receive directly to the data member they are initializing or setting. Standard Library functions like `make_shared` "perfect-forwards" its arguments to the class constructor of whatever object the to-be-created `shared_ptr` is to point to.

The idiomatic expression of the perfect forwarding makes use of the `std::forward`

```
1 template <typename T>
2 void wrapper(T&& arg)
3 {
4     wrapped(std::forward<T1>(t1), std::forward<T2>(t2));
5 }
```

7.2.4 auto

7.2.5 decltype

7.2.6 const-expr

7.3 C++23 New Features

7.3.1 Containers

With C++23, we have 12 associative Containers:

C++98	<code>std::map</code> , <code>std::set</code> , <code>std::multimap</code> , <code>std::multiset</code>
C++11	<code>std::unordered_map</code> , <code>std::unordered_set</code> <code>std::unordered_multimap</code> , <code>std::unordered_multiset</code>
C++23	<code>std::flat_map</code> , <code>std::flat_set</code> , <code>std::flat_multimap</code> , <code>std::flat_multiset</code>

Associative Container	Sorted	Value	Several Identical keys possible	Access Time
<code>std::set</code>	yes	no	no	logarithmic
<code>std::unordered_set</code>	no	no	no	constant
<code>std::map</code>	yes	yes	no	logarithmic
<code>std::unordered_map</code>	no	yes	no	constant
<code>std::multiset</code>	yes	no	yes	logarithmic
<code>std::unordered_multiset</code>	no	no	yes	constant
<code>std::multimap</code>	yes	yes	yes	logarithmic
<code>std::unordered_multimap</code>	no	yes	yes	constant

OOP and UML

8.1 Object Oriented Programming

8.1.1 Polymorphism

In C++ there are different kinds of polymorphism, summarized in the figure 24

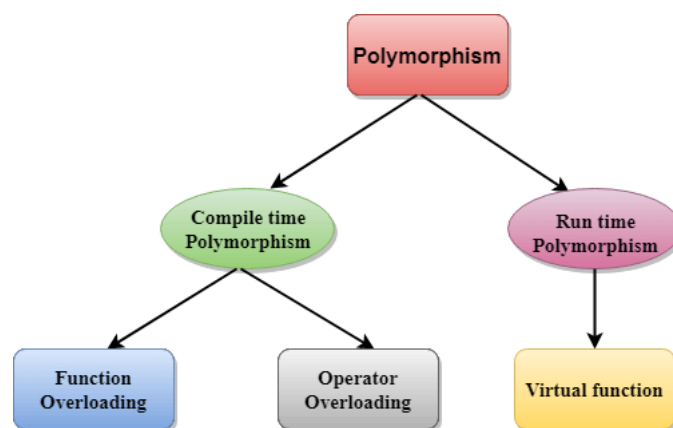


Figure 24: Polymorphism in C++

Compile time polymorphism

Overloaded functions are invoked by matching the type and number of arguments at compile time, because the compiler already has all the information needed. This is known by the name of **static binding** or .

Run time polymorphism

8.2 UML

8.2.1 Class Diagram

In Unified Modeling Language, a class diagram is used to describe the static structure of a system by showing the system's classes, their attributes, operations (methods) and the relationship among them.

The purpose of a class diagram is the conceptual modeling of the structure of the software application. In case of detailed modeling, it is also possible to translate the diagram into programming code.

Visibility

To indicate visibility for methods and attributes, this notation must be placed before their names

- + public
- private
- # protected
- ~ package

Scope

UML defines **instance** and **class** scopes for members. The latter is represented by underlined names.

- **instance members** are scoped to a specific instance
- **class members** are the one commonly recognized as "static"

Arrows

The relationship defined in UML are represented by the following logical connectors

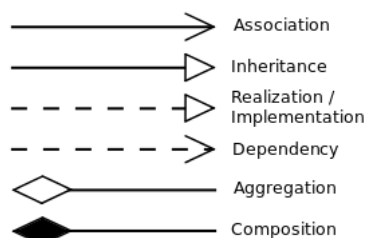


Figure 25: UML logical connectors

- **Dependency** is a relationship which shows that an element requires other model elements for their specification or implementation.
The dashed line points from the dependent (client) to the independent (supplier) element, specifying the direction of a relationship and not of a process.

- **Association** defines a relationship between classes of objects that allows one object instance to cause another to perform an action on its behalf. This relationship is structural and does not represent any behavioural trait.

Code implementation usually requires the requesting object to invoke a method or member function using a **reference** or a **pointer** to the controlled object.

Association can be **bi-directional**, **uni-directional**, aggregation and reflexive.

Objects related via the association are considered to act in a role with respect to the association.

The ends of the association can have all the characteristics of a property: multiplicity, name, visibility and a specification explaining whether the association is ordered and / or unique.

- **Aggregation** is a variant of "**has a**" association relationship and more specific than the latter. It represents a part-whole or a part-of relationship. As a type of association, an aggregation can be named and have the same adornments an association can.

An aggregation can occur when a class is a collection or a container of other classes, but the contained classes do not have a strong lifecycle dependency on the container. **The content of the container continues to exist when the container is destroyed.**

In UML notation, the hollow diamond shape is drawn on the containing class end of the connecting line. An example of aggregation is a Pond which has zero or more Ducks and Duck which has at most one Pond. Duck can exist separately from a pond.

- **Composition** refers to combining objects within compound objects, ensuring the encapsulation of each object by using the well-defined interface without exposing the internals. Differently, data structures do not enforce encapsulation.

Composition can be regarded as a relationship between types: an object of composite type "**has**" objects of other types

Composition must be distinguished from subtyping, which is the process of adding detail to a general data type to create a more specific one.

In UML notation, the filled diamond shape is drawn on the containing class end of the connecting line. An example of composition is a University which has one or more Departments, which exactly one University they belong to. A Department can not exist as a separate entity detached from a specific University.

Class-level relationships

- **Generalization/Inheritance** indicates the one of the two related classes (called **subclass**) is considered to be a specialization from the other (called **super type**). Therefore any instance of the subtype is also an instance of the superclass. An example comes from biological classification: humans are a subclass of simian

which is a subclass of mammal.

Generalization is also known as **inheritance** or a "**is a**" relationship.

The base class is also known as parent, superclass, base class or base type.

The subtype is also known as child, subclass, derived class, derived type, inheriting class or inheriting type.

- **Realization/Implementation** is a relationship between two model elements: one model element (called client) realizes (implements or executes) the behavior that the other model element (the supplier) specifies.

General relationship

- **Dependency** can be a weaker form of bond that indicates that one class depends on another because it uses it at some point in time. Sometimes this relationship might be implemented as member function arguments.
- **Multiplicity** is an optional notation which can be useful to add details to the association relationship. At each end of the line, one of the following notation can be used to indicate the range of number of objects that participate in the association from the perspective of the other end.

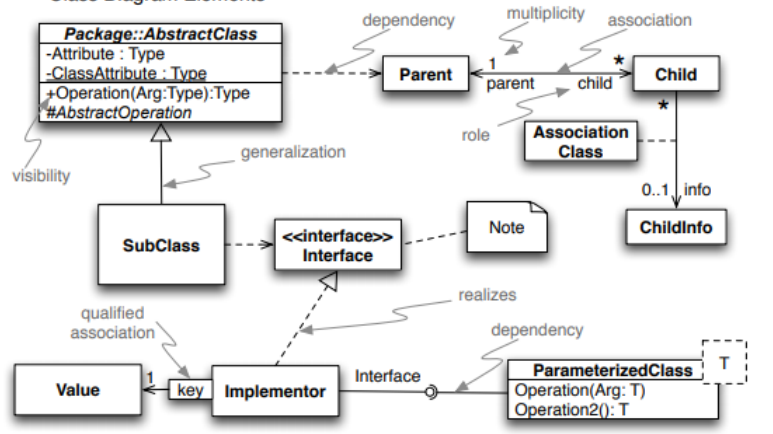
0	No instances
0..1	No instances, or one instance
1	Exactly one instance
1..1	Exactly one instance
0..*	Zero or more instances
*	Zero or more instances
1..*	One or more instances

Class and Sequence diagram Cheatsheet

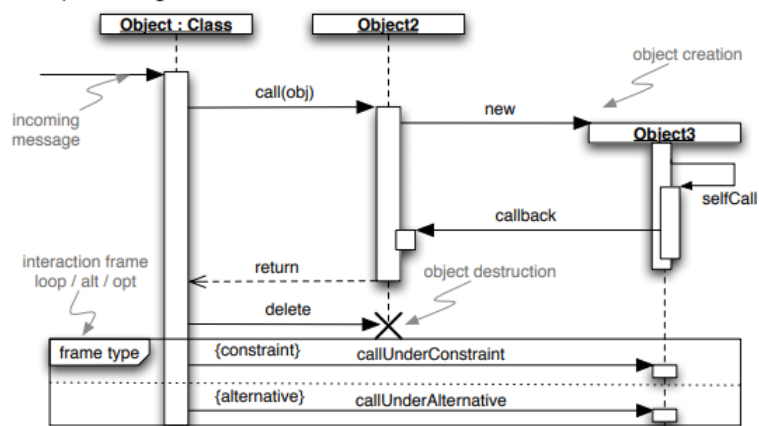
Following a summary of the information for class and sequence UML diagrams

UML Cheatsheet

Class Diagram Elements



Sequence Diagram Elements



(cc) 2006 Lou Franco - Some Rights Reserved - Attribution-NonCommercial-ShareAlike 2.5
<http://creativecommons.org/licenses/by-nc-sa/2.5/>

Figure 26: Class and Sequence diagrams cheatsheet

9

SECTION

Concurrency

C++ includes built-in support for threads, atomic operations, mutual exclusion, condition variables, and futures.

9.1 Threads

<code>thread</code>	manages a separate thread
<code>jthread</code>	thread with support for auto-joining and cancellation

Functions managing the current thread

<code>yield</code>	suggests that the implementation reschedule execution of threads
<code>get_id</code>	returns the thread id of the current thread
<code>sleep_for</code>	stops the execution of the current thread for a specified time duration
<code>sleep_until</code>	stops the execution of the current thread until a specified time point

9.2 Atomic operations

These components are provided for fine-grained atomic operations allowing for lockless concurrent programming. Each atomic operation is indivisible with regards to any other atomic operation that involves the same object. Atomic objects are [free of data races](#).

Atomic types

<code>atomic</code>	atomic class template and specializations for <code>bool</code> , integral and pointer types
<code>atomic_ref</code>	provides atomic operations on non-atomic objects

Operations on atomic types

<code>atomic_is_lock_free</code>	checks if the atomic type's operations are lock-free
<code>atomic_store</code> <code>atomic_store_explicit</code>	atomically replaces the value of the atomic object with a non-atomic argument
<code>atomic_load</code> <code>atomic_load_explicit</code>	atomically obtains the value stored in an atomic object
<code>atomic_exchange</code> <code>atomic_exchange_explicit</code>	atomically replaces the value of the atomic object with non-atomic argument and returns the old value of the atomic
<code>atomic_compare_exchange_weak</code> <code>atomic_compare_exchange_weak_explicit</code> <code>atomic_compare_exchange_strong</code> <code>atomic_compare_exchange_strong_explicit</code>	atomically compares the value of the atomic object with non-atomic argument and performs atomic exchange if equal or atomic load if not
<code>atomic_fetch_add</code> <code>atomic_fetch_add_explicit</code>	adds a non-atomic value to an atomic object and obtains the previous value of the atomic
<code>atomic_fetch_sub</code> <code>atomic_fetch_sub_explicit</code>	subtracts a non-atomic value from an atomic object and obtains the previous value of the atomic
<code>atomic_fetch_and</code> <code>atomic_fetch_and_explicit</code>	replaces the atomic object with the result of bitwise AND with a non-atomic argument and obtains the previous value of the atomic
<code>atomic_fetch_or</code> <code>atomic_fetch_or_explicit</code>	replaces the atomic object with the result of bitwise OR with a non-atomic argument and obtains the previous value of the atomic
<code>atomic_fetch_xor</code> <code>atomic_fetch_xor_explicit</code>	blocks the thread until notified and the atomic value changes
<code>atomic_wait</code> <code>atomic_wait_explicit</code>	replaces the atomic object with the result of bitwise XOR with a non-atomic argument and obtains the previous value of the atomic
<code>atomic_notify_one</code>	notifies a thread blocked in <code>atomic_wait</code>
<code>atomic_notify_all</code>	notifies all threads blocked in <code>atomic_wait</code>

Flag type and operations

<code>atomic_flag</code>	the lock-free boolean atomic type
<code>atomic_flag_test_and_set</code> <code>atomic_flag_test_and_set_explicit</code>	atomically sets the flag to true and returns its previous value
<code>atomic_flag_clear</code> <code>atomic_flag_clear_explicit</code>	atomically sets the value of the flag to false
<code>atomic_flag_test</code> <code>atomic_flag_test_explicit</code>	atomically returns the value of the flag
<code>atomic_flag_wait</code> <code>atomic_flag_wait_explicit</code>	blocks the thread until notified and the flag changes
<code>atomic_flag_notify_one</code>	notifies a thread blocked in <code>atomic_flag_wait</code>
<code>atomic_flag_notify_all</code>	notifies all threads blocked in <code>atomic_flag_wait</code>

Initialization

<code>atomic_init</code>	non-atomic initialization of a default-constructed atomic object
<code>ATOMIC_VAR_INIT</code>	constant initialization of an atomic variable of static storage duration
<code>ATOMIC_FLAG_INIT</code>	initializes an <code>std::atomic_flag</code> to false

Memory synchronization ordering

<code>memory_order</code>	defines memory ordering constraints for the given atomic operation
<code>kill_dependency</code>	removes the specified object from the <code>memory_order_consume</code> dependency tree
<code>atomic_thread_fence</code>	generic memory order-dependent fence synchronization primitive
<code>atomic_signal_fence</code>	fence between a thread and a signal handler executed in the same thread

Mutual exclusion

Mutual exclusion algorithms prevent multiple threads from simultaneously accessing shared resources. This prevents data races and provides support for synchronization between threads.

<code>mutex</code>	provides basic mutual exclusion facility
<code>timed_mutex</code>	provides mutual exclusion facility which implement locking with a timeout
<code>recursive_mutex</code>	provides mutual exclusion facility which can be locked recursively by the same thread
<code>recursive_timed_mutex</code>	provides mutual exclusion facility which can be locked recursively by the same thread and implements locking with a timeout
<code>shared_mutex</code>	provides shared mutual exclusion facility
<code>shared_timed_mutex</code>	provides shared mutual exclusion facility and implements locking with a timeout

Generic mutex management

<code>lock_guard</code>	implements a strictly scope-based mutex ownership wrapper
<code>scoped_lock</code>	deadlock-avoiding RAII wrapper for multiple mutexes
<code>unique_lock</code>	implements movable mutex ownership wrapper
<code>shared_lock</code>	implements movable shared mutex ownership wrapper
<code>defer_lock_t</code> <code>try_to_lock_t</code> <code>adopt_lock_t</code>	tag type used to specify locking strategy
<code>defer_lock</code> <code>try_to_lock</code> <code>adopt_lock</code>	tag constants used to specify locking strategy

Generic locking management

<code>try_lock</code>	attempts to obtain ownership of mutexes via repeated calls to <code>try_lock</code>
<code>lock</code>	locks specified mutexes, blocks if any are unavailable

Call once

<code>once_flag</code>	helper object to ensure that <code>call_once</code> invokes the function only once
<code>call_once</code>	invokes a function only once even if called from multiple threads

Condition variables

<code>condition_variable</code>	provides a condition variable associated with a <code>std::unique_lock</code>
<code>condition_variable_any</code>	provides a condition variable associated with any lock type
<code>notify_all_at_thread_exit</code>	schedules a call to <code>notify_all</code> to be invoked when this thread is completely finished
<code>cv_status</code>	lists the possible results of timed waits on condition variables

Futures

<code>promise</code>	stores a value for asynchronous retrieval
<code>packaged_task</code>	packages a function to store its return value for asynchronous retrieval
<code>future</code>	waits for a value that is set asynchronously
<code>shared_future</code>	waits for a value (possibly referenced by other futures) that is set asynchronously
<code>async</code>	runs a function asynchronously (potentially in a new thread) and returns a <code>std::future</code> that will hold the result
<code>launch</code>	specifies the launch policy for <code>std::async</code>
<code>future_status</code>	specifies the results of timed waits performed on <code>std::future</code> and <code>std::shared_future</code>

Future errors

<code>future_error</code>	reports an error related to futures or promises
<code>future_category</code>	identifies the future error category
<code>future_errc</code>	identifies the future error codes

9.3 Memory Order

- **memory_order_relaxed**

Relaxed operation: there are no synchronization or ordering constraints imposed on other reads or writes, only this operation's atomicity is guaranteed

- **memory_order_consume**

A load operation with this memory order performs a consume operation on the affected memory location: no reads or writes in the current thread dependent on the value currently loaded can be reordered before this load. Writes to data-dependent variables in other threads that release the same atomic variable are visible in the current thread. On most platforms, this affects compiler optimizations only

- **memory_order_acquire**

A load operation with this memory order performs the acquire operation on the affected memory location: no reads or writes in the current thread can be reordered before this load. All writes in other threads that release the same atomic variable are visible in the current thread

- **memory_order_release**

A store operation with this memory order performs the release operation: no reads or writes in the current thread can be reordered after this store. All writes in the current thread are visible in other threads that acquire the same atomic variable and writes that carry a dependency into the atomic variable become visible in other threads that consume the same atomic

- **memory_order_acq_rel**

A read-modify-write operation with this memory order is both an acquire operation and a release operation. No memory reads or writes in the current thread can be reordered before the load, nor after the store. All writes in other threads that release the same atomic variable are visible before the modification and the modification is visible in other threads that acquire the same atomic variable.

- **memory_order_seq_cst**

A load operation with this memory order performs an acquire operation, a store performs a release operation, and read-modify-write performs both an acquire operation and a release operation, plus a single total order exists in which all threads observe all modifications in the same order.

Dictionary

10.1 A

Atomic • Objects of atomic types are the only C++ objects that are free from data races; that is, if one thread writes to an atomic object while another thread reads from it, the behavior is well-defined. In addition, accesses to atomic objects may establish inter-thread synchronization and order non-atomic memory accesses as specified by `std::memory_order`.

Allocate • to assign, allot, distribute, or set apart for a particular purpose

10.2 B

10.3 C

constexpr • a C++ specifier which declares that it is possible to evaluate the value of the function or variable at compile time.

10.4 D

data race •

deadlock • any situation in which no member of some group of entities can proceed because each waits for another member, including itself, to take action, such as sending a message or, more commonly, releasing a lock

duck typing • a concept related to *dynamic typing*, where the type or the class of an object is less important than the methods it defines

10.5 H

Heap • a large pool of memory used to satisfy memory request by allocating part.

10.6 I

Idiom • a group of code fragments sharing an equivalent semantic role (meaning), which recurs frequently across software projects often expressing a special feature in one or more programming languages or libraries.

10.7 M

method chaining •

10.8 O

Overloading • a compile time polymorphism achieved specifying more than one function with the same name but different signature in the same scope.

Overriding • the act of redefining a method of a base class in a derived class, using the same name, return type and parameters.

10.9 P

Polymorphysm • ability of different objects to be accessed by a common interface.

10.10 R

Race Condition • undesirable situation which occurs when two instructions from different threads access the same memory location, at least one of these accesses is a write and there is no synchronization that is mandating any particular order among these accesses.

RAII • acronym for Resource Acquisition Is Initialization.

RVO • acronym for Return Value Optimization.

10.11 S

Stack • LIFO-ordered contiguous region of temporary memory whose size is known to the compiler, which manages to allocate and deallocate blocks of memory for the stack variables like functions' local data. . If a region of memory lies on the thread's stack, that memory is said to have been allocated on the stack, i.e. stack-based memory allocation (SBMA).

References

- [1] Robert C. Martin (2014) - *Principles of OOD*
Retrieved from: [The OLD Object Mentor blog site](#)
- [2] Robert C. Martin (2009) - *Getting a SOLID start*
Retrieved from: [Uncle Bob Consulting LLC](#).
- [3] Sandu Metz (2009) - *SOLID Object-Oriented*
Retrieved from: [Ghost Archive - Gotham Ruby Conference](#)
- [4] Robert C. Martin (2000) - *Design Principles and Design Patterns*
Retrieved from: [Internet Archive - Objectmentor](#)
- [5] Robert C. Martin (2015) - *Single Responsibility Principle*
Retrieved from: [Internet Archive - Objectmentor](#)
- [6] Robert C. Martin (2003) - *Agile Software Development, Principles, Patterns, and Practices*
Retrieved from: [Internet Archive - Objectmentor](#)
- [7] *Open/Closed Principle*
Retrieved from: [Internet Archive - Objectmentor](#)
- [8] *Liskov Substitution Principle*
Retrieved from: [Internet Archive - Objectmentor](#)
- [9] *Interface Segregation Principle*
Retrieved from: [Internet Archive - Objectmentor](#)
- [10] *Dependency Inversion Principle*
Retrieved from: [Internet Archive - Objectmentor](#)
- [11] *Clean Architecture: A Craftman's Guide to Software Structure and Design*
Retrieved from:
<https://books.google.com/books?id=uGE1DwAAQBAJ&q=2004+or+thereabouts+by+Michael+Feathers>
- [12] *Pimpl Idiom*
Retrieved from: [Wiki Wiki Web](#)
- [13] Herb Sutter - *The Fast Pimpl Idiom*
Retrieved from: [Guru of the Week - 28](#)
- [14] Herb Sutter - *Compilation Firewalls*
Retrieved from: [Guru of the Week - 24](#)
- [15] Herb Sutter - *Compilation Firewalls (C++11-updated version of GotW #24)*
Retrieved from: [Guru of the Week - 100](#)

- [16] Herb Sutter - *The Joy of Pimpls (or more about the Compiler-Firewall Idiom)*.
Retrieved from: [More About Compiler Firewalls](#).
- [17] *What is the copy and swap idiom?*
Retrieved from: [Stackoverflow](#).
- [18] Herb Sutter - *Exception-Safe Class Design, Part I: Copy Assignment*
Retrieved from: [Guru of the Week - 59](#)
- [19] Herb Sutter - *Exception-Safe Class Design, Part II: Inheritance*
Retrieved from: [Guru of the Week - 60](#)
- [20]
Retrieved from:
- [21] *C++ Core Guidelines*
Retrieved from: [Standard Cpp Foundation Github](#).
- [22] *Memory Management: Stack and Heap*
Retrieved from: [CppCon YouTube Video](#)
- [23] *Leak-freedom in C++ ... By Default - CppCon 2016*
Retrieved from: [CppCon YouTube Video](#)
- [24] R. Martinho Fernandes (2012) - *Rule of Zero*
Retrieved from:
- [25] Scott Meyers (2014) - *A Concern about the Rule of Zero*
Retrieved from:
- [26] *A polymorphic class should suppress public copy/move*
Retrieved from: [Standard Cpp Foundation Github](#)
- [27] *If you define or =delete any copy, move, or destructor function, define or =delete them all*
Retrieved from: [Standard Cpp Foundation Github](#)
- [28] *References*
Retrieved from: [ISOC++ Wiki](#)
- [29] *Const Correctness*
Retrieved from: [ISOC++ Wiki](#)
- [30] (2016) - *Core C++ - lvalues and rvalues*
Retrieved from: [Just Software Solutions](#)
- [31] (2008) - *Rvalue References and Perfect Forwarding*
Retrieved from: [Just Software Solutions](#)

- [32] Peter Dimov, Howard E. Hinnant, Dave Abrahams - *The Forwarding Problem: Arguments*
Retrieved from: [open-std.org - N1385=02-0043](https://open-std.org/N1385=02-0043)
- [33] Meyers, Sutter, Alexandrescu (2011) - *session-announcement-adventures-in-perfect-forwarding*
Retrieved from:
- [34] *What are the main purposes of std::forward and which problems does it solve?*
Retrieved from: [StackOverflow Thread](#)
- [35] *What is the move semantic?*
Retrieved from: [StackOverflow Thread](#)
- [36] Howard Hinnant (2014) - *Everything you wanted to know about Move Semantics*
Retrieved from: [Slideshare](#)
- [37] *If your function must not throw declare it noexcept*
Retrieved from: [Cpp Core Guidelines](#)
- [38] *Object-Oriented Programming Using C++*
Retrieved from: [Weber State University - School of Computing - Computer Science](#)
- [39] *Allen Holub's UML Quick Reference*
Retrieved from: [Allen Holub website](#)
- [40] *Class Diagram*
Retrieved from: [WikiPedia - Class Diagram](#)
- [41] *State Diagram*
Retrieved from: [WikiPedia - State Diagram](#)
- [42] *State Machine*
Retrieved from: [WikiPedia - State Machine](#)
- [43] *Programming paradigm*
Retrieved from: [WikiPedia - State Machine](#)
- [44] *Composition over Inheritance*
Retrieved from: [WikiPedia - Composition over Inheritance](#)
- [45] *Delegation (object-oriented programming)*
Retrieved from: [WikiPedia - Delegation](#)
- [46] *Role-oriented programming*
Retrieved from: [WikiPedia - Programming paradigm](#)
- [47] James Rumbaugh (1991) - *Object-Oriented Modeling and Design*
Retrieved from: [Internet Archive](#)

- [48] Herb Sutter - *Object Lifetimes - Part I*
Retrieved from: [Guru of the Week - 22](#)
- [49] Herb Sutter - *Object Lifetimes - Part II*
Retrieved from: [Guru of the Week - 23](#)